

5 as shown below:

```
(-when-let variable-value body) ;; Syntax
(-when-let (match-index (string-match "p" "lisp")) (+ match-index 2))
5
```

The *-when-let** API binds all the values to the respective variables, and if all of them are evaluated as *True*, then the body is executed. For example:

```
(-when-let* variable-values body) ;; Syntax

(-when-let* ((x 1) (y 2) (z (+ y 4))) (+ x y z))
9
```

The *-if-let* function evaluates the variable-value and if the value returned is non-nil, the *then* expression is evaluated; otherwise, the *else* expression is executed as shown below:

```
(if-let variable-value then else) ;; Syntax

(if-let (match-index (string-match "x" "lisp")) (+ match-index 2) 0)
0
```

The *-if-let** function is similar to the *-if-let* function, except that it takes variable-values as arguments, along with the *then* and *else* expressions. An example is given below:

```
(-if-let* variable-values then else) ;; Syntax

(-if-let* ((x 1) (y 2) (z 3)) (+ x y z) "False")
6
```

Destructive operations: The following two functions modify a list. The *!cons* function sets the *cdr* to the cons of the input arguments, as shown below:

```
(!cons car cdr) ;; Syntax

(let (x) (!cons 5 x) x)
(5)
```

The *!cdr* function sets the list to the *cdr* of the input list. For example:

```
(!cdr list) ;; Syntax

(let ((x '(1 2))) (!cdr x) x)
(2)
```

Function combinators: The *dash-functional* library is

required to use the function combinators. You need to add the following to your Emacs initialisation configuration file in order to use the functions:

```
(require 'dash-functional)
```

The *-partial* function takes a function and a list of arguments. The input function takes fewer arguments and returns a function that accepts and evaluates the additional arguments. An example is shown below:

```
(-partial function arguments) ;; Syntax

(funcall (-partial '- 5) 2)
3
```

The *-rpartial* function is similar to the *-partial* function, except that the additional arguments are used as the first argument on the returned partial function. For example:

```
(-rpartial function arguments) ;; Syntax

(funcall (-rpartial '- 5) 2)
-3
```

www.efy.in

EFYGROUP
Technology Drives Us

“
Nothing except
the mint can make
money without
advertising.”
”



-Thomas
Babington Macaulay

